

RESEARCH

Open Access



EnLeM: ensemble learning-based model to detect phishing websites

Most Nilufa Yeasmin¹, Md Abu Rumman Refat², Bikash Chandra Singh^{3*}, Zulfikar Alom⁴, Zeyar Aung⁵ and Mohammad Azim⁶

Abstract

Phishing involves manipulating individuals into revealing private data, e.g., user IDs, bank details, and passwords. The observed surge in fraud is related to increased deception, impersonation, and advanced online attacks. Thus, effective phishing detection methods are required to mitigate escalating global phishing threats. Existing methods (e.g., heuristics-based, signature-based, and visual similarity-based methods) attempt to detect phishing sites, and machine learning (ML) and deep learning (DL) methods are effective in the cybersecurity context in terms of learning from data, offering insights, and forecasting. However, independent ML algorithms are limited when handling complex data, and DL techniques surpass traditional ML methods in terms of performance but require more data and time. To tackle these challenges, we present EnLeM, an ensemble learning model designed specifically for phishing website detection. EnLeM brings together three well-known machine learning classifiers—decision tree, random forest, and k-nearest neighbor—using a hard voting mechanism, and further strengthens efficiency with Mutual Information-based feature selection. When tested on the UCI phishing dataset, EnLeM delivered strong results, reaching 97.21% accuracy and a 97.51% F1-score. Compared to individual ML classifiers, it consistently performed better, and it also proved more efficient than deep learning models such as CNN and LSTM. Notably, EnLeM maintained stable accuracy across different feature subsets while cutting execution time by roughly 13%. By striking a balance between accuracy, speed, and interpretability, EnLeM stands out as a practical and scalable solution for real-time phishing detection without the heavy resource demands of deep learning approaches.

Keywords Phishing, Univariate feature selection, Machine learning, Ensemble algorithm, Random forest, K-NN, Cybersecurity

1 Introduction

With widespread Internet access and the ongoing emergence of innovative services, such as web 3.0, most individuals have transitioned to online platforms to perform a wide range of tasks, e.g., shopping and banking, thereby minimizing the need for physical queues. However, cybercriminals can exploit this trend to obtain funds illicitly, and the prevalence of phishing kits simplifies this process, enabling those with limited technical expertise to initiate effective phishing campaigns. Phishing, which is a social engineering attack, involves scammers masquerading as trustworthy sources to extract personal information by connecting through e-mails, telephone calls, or text messages to access victims' financial

*Correspondence:

Bikash Chandra Singh
bsingh@odu.edu

¹ Computer Science, Bangladesh University of Business and Technology, Dhaka 1216, Bangladesh

² Computer Science and Engineering, Green University of Bangladesh, Dhaka 1216, Bangladesh

³ School of Cybersecurity, Old Dominion University, Norfolk 23529, VA, USA

⁴ Department of Computer Science, University of Manitoba, Winnipeg R3T 2N2, Canada

⁵ Center for Secure Cyber-Physical Systems (C2PS) and Department of Computer Science, Khalifa University, Abu Dhabi 127788, UAE

⁶ Department of Computer Science, Asian University for Women, Chattogram 4000, Bangladesh

accounts. The occurrence of phishing attacks is on the rise, with attackers observing victims' online actions and bypassing security measures [1, 2]. The American Federal Bureau of Investigation identified phishing as the primary cybercrime in 2020, and instances of repeated attacks have increased. Monthly phishing attacks surged by 65%, spawning 1.5 million sites in 2018 [3], and a notable breach exposed data for 80 million Anthem Healthcare customers in 2014 [4].

Numerous approaches have been proposed for phishing detection. The predominant methods to identify phishing are the *heuristics-based approach* and the *signature-based method*. The heuristics-based approach is a conventional means of identifying fraudulent web pages, and it is primarily used to identify unauthorized sites [5]. Despite its accuracy-enhancing role, the heuristics-based approach is intricate and costly to implement [6]. Similarly, the signature-based method is a critical tool for identifying phishing sites using a database containing blacklisted phishing sites. However, the signature-based method fails to identify newly generated phishing sites [7].

Another method to detect phishing websites involves the *visual similarity-based approach*, which relies on an assortment of images, logos, text formatting, and HTML tags from a database. However, this method requires substantial computational resources to realize effective image processing [6].

Machine learning (ML) can also be used to identify phishing websites. Despite the various ML techniques employed to distinguish between phished and legitimate sites, conventional ML models frequently fall short in detecting phishing webpages as effectively as browser security indicators. For example, only 23% of these models rely on webpage content to determine legitimacy [8]. In addition, the general population may struggle to differentiate between the padlock icon in a browser and the actual webpage content, and ML-based detection methods incur considerable computational costs, and their sluggish processing can contribute to increased latency when identifying phishing websites [6]. Previous studies have highlighted other limitations of contemporary ML models, including increased false positives, decreased detection rates, and classifier inefficiencies [9, 10]. Many traditional ML models also struggle with unstructured and streaming data due to the diverse nature of real-world data, which frequently contain noise, irrelevant attributes, redundant features, and anomalies [11].

Other studies have investigated methods to identify malicious webpages automatically using *deep learning (DL) models*. However, the performance of DL-based classification methods, which is influenced by various factors, e.g., the available training data and features,

varies across different algorithms. Numerous features can enhance the performance of DL models; however, their slower speed can hinder efficient phishing website detection. In addition, training DL models requires substantial data and resources, e.g., multiple GPUs and parallel processing to expedite the learning and classification phases [12].

Due to the limitations of current models, including heuristics-based approaches, signature-based methods, visual similarity-based approaches, ML-based methods, and DL-based methods, in identifying phishing websites effectively and efficiently, expecting users to differentiate phishing sites from legitimate sites accurately in real-time is impractical. Ensemble learning is a common ML technique that utilizes multiple ML models in a cooperative manner to produce better results. Here, by leveraging the strengths of different models and compensating for their weaknesses, the overall outcome can be improved. Several types of ensemble models have been proposed, e.g., bagging, boosting, and stacking models. The outcomes are frequently predicted through a voting mechanism, either hard voting or soft voting. Ensemble learning methods have been employed successfully in a wide range of fields, e.g., medical imaging [13], social media [14], finance [15], and cybersecurity [16].

This study introduces a lightweight hard-voting ensemble for phishing website detection that integrates decision tree, random forest, and k-nearest neighbor, designed to balance accuracy and execution time. Mutual Information-based SelectKBest is embedded *within* cross-validation to reduce dimensionality without data leakage, yielding stable performance while lowering runtime. An ablation on the top-*k* threshold demonstrates robustness to noisy or redundant attributes. A comprehensive benchmark against five ML and two DL baselines shows that the proposed approach attains competitive accuracy and F1 with lower computational cost than DL models. Implementation details include a transparent tie-breaking rule for the 1–1–1 case and hyperparameter optimization via grid search with 10-fold CV, improving reproducibility and fairness of comparison.

The primary contributions of this study are summarized as follows:

1. The proposed EnLeM, a hard-voting ensemble that integrates Decision Tree, Random Forest, and k-Nearest Neighbor. Unlike earlier ensemble models such as Ubung et al. [17] and Toolan et al. [18], which primarily improved accuracy without addressing computational efficiency, EnLeM jointly optimizes accuracy and execution time while maintaining robustness.

2. Prior studies (e.g., Salihovic et al. [19] highlighted the negative impact of noisy and redundant features on phishing detection. To overcome this, we apply Mutual Information-based SelectKBest feature selection within cross-validation, reducing dimensionality, cutting execution time (10.01 s $\rightarrow$ 8.72 s), and preserving accuracy. Unlike CNN and LSTM baselines [20], our model remains stable when features are reduced.
3. EnLeM achieves high accuracy (97.21%) and F1-score (97.51%) with a low error rate (0.0279). This demonstrates that a lightweight ensemble can rival deep learning models in detection quality, while avoiding their heavy computational cost.
4. We evaluate EnLeM against five machine learning algorithms and two deep learning approaches. Results show that EnLeM consistently outperforms ML baselines and offers better efficiency than DL methods, highlighting its practicality as a scalable solution for real-time phishing detection.

The remainder of this paper is organized as follows. First, a brief outline of related work is presented in Sect. 2. Section 3 presents the proposed EnLeM architecture for phishing website detection. Section 4 first introduces the performance metrics and experimental design used to evaluate the proposed EnLeM, and the experimental results are presented and analyzed. Finally, the paper is concluded in Sect. 5, including suggestions for potential future work.

2 Related work

Phishers attempt to acquire personally identifiable information, banking information, credit card details, and user passwords. Phishing websites are generated to mimic legitimate websites in order to obtain personal information from users. For example, cybercriminals can gain access to relevant personal data via e-mails, text messages, direct messages on social media, or through video games.

Many phishing detection methods have been proposed previously, including signature-based, visual-based, ML-based, and DL-based methods.

For example, Salihovic et al. [19] used artificial neural networks, logistic regression, random forest (RF), SVM, k -nearest neighbor, and naive Bayes methods to detect phishing websites. First, they used the UCI phishing websites dataset, which contains 31 attributes. Then, they applied two feature selection methods, i.e., the BestFirst+CfsSubsEvaluation and the Ranker+Principal Components feature selection optimizer. In an initial experiment, they achieved the highest accuracy (97.33%) using the RF method. Here, the first optimizer reduced

10 attributes in the phishing dataset with an average reduction in accuracy of 1.53%. In a subsequent experiment, they reduced only a single feature. Here, the accuracy of the RF model increased, whereas it decreased by 0.09% for the SVM method. In addition, the RF method obtained the best performance on a spam e-mail dataset.

Yuan et al. [21] proposed a phishing website detection method based on features from URLs and webpage links using the deep forest model, resulting in a positive rate of 98.3% and a false alarm rate of 2.6%. Primarily, they employed an effective strategy based on search operators via search engines to find the phishing targets with an accuracy of 93.98%. In addition, Jain and Gupta [22] shortlisted 19 of the most important features and analyzed several features of phishing web pages. Here, they used the SVM, RF, neural network, logistic regression, and naive Bayes (NB) methods to classify phishing webpages, and they obtained the highest detection accuracy of approximately 99% with RF.

Gu et al. [23] introduced an automatic system to detect phishing web pages using an NB classifier and analyzing the feature URL. They classified using an SVM for ambiguous web pages. They demonstrated that their system obtains high accuracy with low time costs. In addition, Moghimi and Varjani [24] proposed a method that combines SVM and decision tree (DT) models, where each model served distinct purposes. For example, the SVM was used for training, and the DT model was employed to generate the rules used to detect phishing websites targeting the banking field.

In addition, many studies have proposed DL-based models for phishing website detection. For example, Yerima et al. [20] proposed a CNN-based phishing webpage detection method. Here, they used the UCI phishing websites dataset of 6,157 genuine and 4,898 phishing websites, and they compared their proposed CNN model with conventional ML models. The results demonstrated that their model achieved the highest accuracy.

Zhang et al. [25] proposed multilayer perceptron neural networks to detect phishing e-mails, and they evaluated the effectiveness and efficiency of their model. They compared the proposed model with different classification algorithms, e.g., NN, SVM, DT, and NB algorithms. They found that their model obtained the highest accuracy and recall values, approximately 95%, and they demonstrated that their model was one of the best in terms of detecting phishing e-mails. In addition, Rao and Pais [26] proposed a classification model based on a heuristic dataset. They trained eight ML models, including DT (J48), RF, logistic regression, Bayes network, multilayer perceptron, sequential minimal optimization (SMO), AdaBoostM1, and SVM models, and they demonstrated that the RF algorithm outperformed the other models with

an accuracy of 99.31%. They also attempted to identify the best classifier for phishing detection using different (orthogonal and oblique) RF classifiers. They found that the principal component analysis RF (PCA-RF) method obtained the highest accuracy of approximately 99.55%.

Aksu et al. [27] studied various traditional ML and DL approaches (e.g., the neural networks, SVM, DT, DNN, and stacked auto-encoder technique approaches) for the phishing detection task. They proved that the DL technique performed the best in terms of phishing detection accuracy (80%).

In addition, Zhang et al. [28] proposed a model to detect Chinese e-commerce phishing websites based on the URL and website content. They included 15 domain-specific features to detect phishing attacks, and they evaluated the websites with four traditional ML models, i.e., the SMO, NB, RF, and logistic regression models. The experimental results demonstrated that the SMO algorithm obtained the best phishing detection performance, with accuracy and F1-score results of 95.83% and 95.58%, respectively. However, their experiment targeted a single domain of phishing sites, i.e., it was ineffective with non-Chinese websites.

Khan et al. [29] proposed the *BDLT-IoMT* architecture, which integrates support vector machines (SVM) with blockchain technology to secure data processing in Internet of Medical Things (IoMT) environments. Their approach emphasizes infrastructure security, seamless peer-to-peer communication, and data integrity across distributed healthcare systems. The model demonstrated improved resilience to cyberattacks and optimized resource management compared to conventional IoMT frameworks. Although healthcare-oriented, this study highlights how combining ML techniques with blockchain-based security can provide useful insights into designing lightweight and trustworthy detection systems.

Whittaker et al. [30] proposed a blacklisting technique based on known phishing websites, and they extracted standard features from their dataset to detect phishing sites using an RF classifier. They showed that this RF-based classifier achieved the best accuracy with noisy live phishing data and obtained a false positive rate of less than 0.1%. However, this method only works effectively if the phishing websites have been published or are visible.

Ubing et al. [17] proposed an ensemble learning (EL) model based on majority voting to detect phishing websites using feature selection techniques. They compared their method with traditional ML models, and the results demonstrated that their model outperformed the current phishing website detection methods, obtaining accuracy and F1-score values of approximately 95.4% and 94.7%, respectively. However, their analysis involved some important limitations. For example, the authors focused

only on improving accuracy; other performance metrics should be considered to be more balanced.

Toolan et al. [18] used the C5.0 DT algorithm and an ensemble of other classifiers (*k*-nearest neighbor, SVMs, NB, and linear regression) to classify e-mails as phishing or nonphishing. However, they only used 8000 e-mails with five features, and their systems classified approximately 50% of the e-mails as phishing, and the remaining 50% were classified as legitimate. The average accuracy and precision values of the C5.0 model were approximately 97.15% and 98.56%, respectively, and the average ensemble accuracy was 93.68%. They also discussed the advantages of recall boosting classifiers, which provided better accuracy than the C5.0 and ensemble classifiers.

Table 1 summarizes these existing phishing detection methods. However, the current study proposed a novel approach based on ML techniques. The method proposed in this paper realizes very efficient phishing website detection compared to the existing methods.

3 Overall system framework

The architecture of the proposed EnLeM architecture is shown in Fig. 1. In this study, we used the UCI dataset [31], which contains phishing and legitimate URLs. Note that we require data preprocessing techniques to improve the performance metrics.

3.1 Dataset

The UCI phishing website dataset [31] was used to evaluate the performance of the proposed EnLeM and existing

Table 1 Summary of existing method phishing website detection methods and the proposed EnLeM method

Research work	Methodology
Salihovic et al. (2019) [19]	ML with feature selection
Yuan et al. (2018) [21]	Deep forest
Jain and Gupta (2018) [22]	ML with feature selection
Gu et al. (2013) [23]	ML
Moghimi and Varjani (2016) [24]	ML
Yerima et al. (2020) [20]	CNN (DL)
Zhang et al. (2012) [25]	MLP neural networks
Aksu et al. (2019) [27]	ML and DL
Zhang et al. (2014) [28]	ML with feature selection
Whittaker et al. (2010) [30]	Blacklist technique
Ubing et al. (2019) [17]	Ensemble learning (EL) with feature selection
Toolan et al. (2009) [18]	C5.0 and EL
Proposed EnLeM method	Majority voting-based EL with ML methods and univariate feature selection

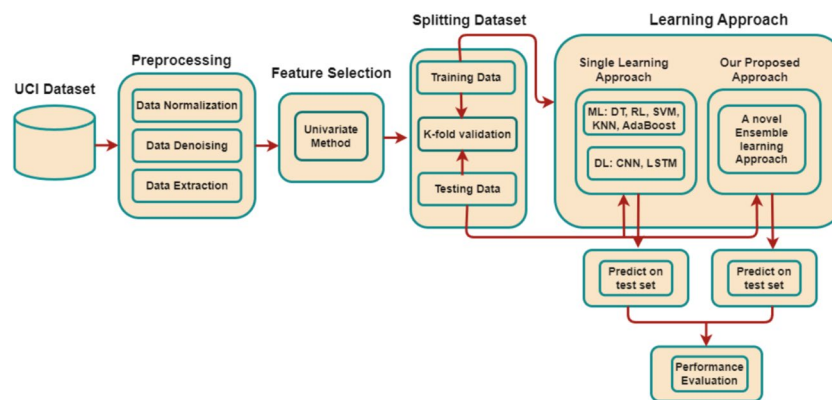


Fig. 1 Overall system framework of proposed EnLeM architecture

models, including six ML-based models and two DL-based algorithms. This dataset is described as follows.

Note that descriptions of the features in this dataset can be found in the literature [31, 32]. The dataset comprises a total of 30 features, with 11,055 instances extracted from 6157 legitimate webpages and 4898 phishing webpages. This dataset was selected for this study because it is recent and available in the public domain. The dataset's attributes and corresponding values are summarized in Table 2.

3.2 Data preprocessing

Realizing sufficient performance with learning algorithms is dependent on the quality of the data used to solve the target classification problem. Thus, data preprocessing is required to achieve better results. Several data preprocessing techniques have been proposed [33, 34]; however, in this study, we applied the data normalization, data denoising, and data extraction techniques. Using data normalization, we can reduce the bias in the attributes in the dataset with higher numerical contributions and obtain all features in a typical range. With data normalization, we can also ensure that all features have an equal numerical contribution; however, not all features are equally important for the classification task. Some features are highly relevant, and others are irrelevant or redundant, which increases the complexity of learning algorithms. In addition, most irrelevant features result in the curse of dimensionality, and we solve this problem by implementing the feature selection method (Sect. 3.3).

We designate a single feature as the target with a label. We assign the label “legitimate” when the result value is 0, and the label “phishing” is assigned if the result value is 1. The extracted features of legitimate and phishing URL datasets in the UCI dataset are concatenated without

Table 2 Feature descriptions of collected dataset

Attribute	Type	Possible values
Having IP address	Numeric	−1, 1
URL length	Numeric	1, 0, −1
Shortening service	Numeric	1, −1
Having ‘at’ symbol	Numeric	1, −1
Double slash redirecting	Numeric	−1, 1
Prefix suffix	Numeric	−1, 1
Having sub domain	Numeric	−1, 0, 1
SSLfinal state	Numeric	−1, 1, 0
Domain registration length	Numeric	−1, 1
Favicon	Numeric	1, −1
Port	Numeric	1, −1
HTTPS token	Numeric	−1, 1
Request URL	Numeric	−1, 1
URL of anchor	Numeric	−1, 0, 1
Links in tags	Numeric	1, −1, 0
SFH (server form handler)	Numeric	−1, 1, 0
Submitting to e-mail	Numeric	−1, 1
Abnormal URL	Numeric	−1, 1
Redirect page	Numeric	0, 1
On mouse over (using to hide link)	Numeric	1, −1
Right click	Numeric	1, −1
Using pop-up window	Numeric	1, −1
Iframe	Numeric	1, −1
Age of domain	Numeric	−1, 1
DNS record	Numeric	−1, 1
Web traffic	Numeric	−1, 0, 1
PageRank	Numeric	−1, 1
Google Index	Numeric	−1, 1
Links pointing to page	Numeric	1, 0, −1
Statistical report	Numeric	−1, 1

shuffling. However, note that we shuffled both the training and testing datasets.

3.3 Feature selection

The feature selection process involves selecting a subset of relevant features or attributes for various reasons, e.g., some features are easier for users to interpret, reducing training time, avoiding unnecessary attributes, and improving data compatibility. Previous studies have utilized a variety of feature selection methods to improve model compatibility, including univariate selection, recursive feature elimination, and principal component analysis.

To prevent data leakage, feature selection was performed independently within each fold of the 10-fold cross-validation. In other words, for every training fold, Mutual Information-based SelectKBest was fitted only on that fold's training data, and the selected features were then applied to the corresponding validation fold. This ensures that no information from the validation data influenced the choice of features.

In this study, we used the univariate feature selection method, which selects the best essential features based on univariate statistical tests. Here, we used the Select- k -Best technique, which selects features according to the k highest scores, and different statistical tests can be used in this selection method. The univariate test uses the mutual information (MI) approach ("*mutual_info_classif*" in the scikit-learn ML library) and selects the top 20 features from the dataset.

The MI between two random variables is a nonnegative value, which measures the dependency between the variables. The MI value is zero if and only if the two random variables are independent; otherwise, the MI value is one. Here, if X and Y denote the two random variables, then we obtain the following:

$$I(X; Y) = H(X) - H(X|Y), \quad (1)$$

where $I(X; Y)$ is the MI of X , Y , $H(X)$ denotes the entropy of X , and $H(X|Y)$ is the conditional entropy for X given Y .

In this study, the MI approach is employed to select the top 20 features from the 30 features in the dataset. Figure 2 shows top 20 features for classification purposes.

3.4 Dataset splitting

To split the UCI dataset, we used the k -fold cross-validation technique, which is a strategy to enhance the single holdout approach. Rather than splitting a dataset into the training and the testing (holdout) sets only once, this technique divides the dataset into k subsets and applies the holdout approach to each subset k times. The steps involved in the k -fold cross-validation process are described as follows.

1. Divide the dataset into k folds or k chunks.
2. Create k folds from the dataset. For each fold, construct a model using the other $k - 1$ folds. Then, we assess the model's performance on the k -th fold by predicting the labels (i.e., phishing or legitimate) and evaluate the accuracy.
3. This procedure is repeated until all k folds have been evaluated using separate test sets.
4. The results are recorded, and the expected average accuracy is calculated and given as the test measure for the model currently being evaluated.

For each of our classifiers, the k -fold cross-validation was considered. Here, we experimented with several k values and determined that $k = 10$ was the best choice for our experiment.

3.5 Proposed architecture

Here, we first define the formal statement for the target phishing website detection problem, and then we describe the overall framework of the proposed EnLeM, including its formal definition.

3.5.1 Problem statement

Here, assume we have a set U of websites, where $U = \{u|u = x_i, x_i \in \text{website}, i \in N^+, |U| = n\}$. X_P indicates phishing, where $X_P = \{x|x = p, p \in \text{Phishing}\}$, and let X_L indicate legitimate, where $X_L = \{x|x = l, l \in \text{Legitimate}\}$. In addition, let U_s be a suspicious website. Formally, the phishing website detection problem can be defined as follows.

Definition 1 (Phishing Website Detection of U_s): Let $P(C)$ be the power set of C , and the time cost of detecting U_s is denoted t_c . The function $t : U \rightarrow P(C)$ represents the mapping relationship required to implement the detection of U_s . Here, let C'_r be the result calculated by t , and $C' = U_{i \in n} C'_r, C_r$ as the category of the website U_s ($C_r \in C$), where:

$$C_r = \begin{cases} 1 \rightarrow \text{Phishing;} \\ 0 \rightarrow \text{Legitimate;} \end{cases}$$

Thus, the phishing website detection problem can be expressed as follows:

$$\forall U_s \in U : C'_r = t(U_s) \quad (2)$$

The objective function is defined as follows.

$$O(U) = \max \left(n - \sum_{i=1}^n (t(U_s) \oplus C_r), \sum_{i=1}^n t_c(U_s) \right) \quad (3)$$

Here, $\oplus$ denotes the XOR operation, and $t_c(U_s)$ represents the time cost associated with the detection of U_s .

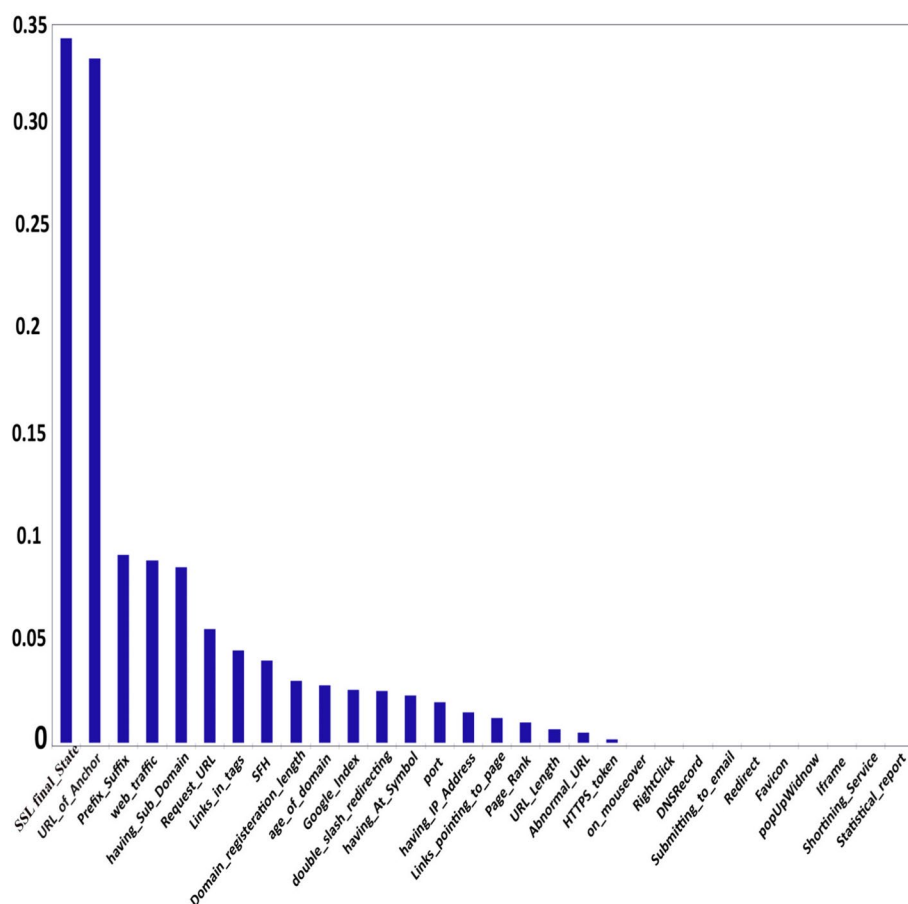


Fig. 2 Graphical representation of MI values of attributes in feature selection

The goal is to find a suitable function t such that the objective function $O(U)$ obtains its maximum value. The goal of the phishing website detection problem is to identify a mapping t that maximizes the objective function.

Phishing websites continue to evolve rapidly, often imitating legitimate services and bypassing blacklist-based detection. Existing machine learning and deep learning models face a trade-off: deep neural networks offer high accuracy but are computationally expensive, while traditional ML classifiers are lightweight but prone to unstable results or high false positives. The specific problem addressed in this work is how to design a lightweight ensemble learning model that achieves competitive accuracy while reducing execution time and maintaining interpretability. The scope of this study is limited to centralized phishing website detection using the publicly available UCI dataset, which contains 11,055 labeled URLs. The evaluation considers classical ML and DL baselines under consistent experimental settings with 10-fold cross-validation. Broader topics such as distributed or federated detection, LLM-driven anomaly

detection, or deployment in real-world browsers are beyond the scope of this paper but are discussed as directions for future work.

3.5.2 Proposed EnLeM framework

We introduce an approach based on hard voting EL that predicts the class labels with the highest sum of the votes from the learning models. The proposed **EnLeM** is an extended voting-based EL approach, as shown in Fig. 3.

Although we considered alternative ensemble strategies such as soft voting and weighted voting, hard voting was selected for its balance of efficiency and stability. Preliminary experiments showed that soft voting improved accuracy by less than 0.3% while increasing inference time by approximately 12%. Weighted voting also required probability calibration, which was unreliable for k-NN and Decision Tree, leading to unstable outputs. Therefore, hard voting was adopted as the most practical aggregation rule for EnLeM, ensuring robust and interpretable results.

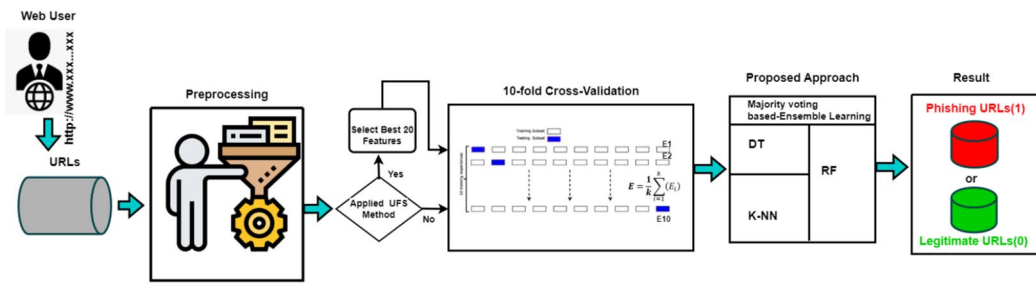


Fig. 3 Proposed phishing website detection framework

When web users browse webpages on the Internet, they frequently lack awareness of whether the website is legitimate or a phishing attempt. To address this issue, we used a dataset containing 11,055 URLs from the UCI phishing website datasets (Sect. 3.1). These include examples of legitimate and phishing websites. Then, the following preprocessing techniques are employed to improve the corresponding performance metrics.

1. Handling missing values.
2. Data cleaning. We remove outliers and duplicates.
3. Feature engineering. We extract irrelevant features and convert categorical labels (“legitimate” and “phishing”) to numerical values.
4. Normalization.
5. Dataset splitting (Sect. 3.4).

After preprocessing the dataset, we employ the proposed majority voting-based model for phishing website detection. Typically, the EL technique combines multiple models that solve a given problem using combined decisions that multiple classifiers provide. Essentially, EL is used to improve the performance of various tasks, e.g., classification, prediction, and function approximation tasks. Numerous applications of EL include decision-making, optimal feature selection, data fusion, incremental learning, nonstationary learning, and error-correcting [35]. Dasarathy et al. [36] were among the first to propose the EL approach in a divide-and-conquer fashion, where the feature space was partitioned using two or more classifiers.

For example, someone planning to purchase a new computer could go to a computer showroom and purchase a machine shown to them by the salesperson. However, most people are likely to ask their friends or colleagues for advice or go to online portals showing relevant reviews. Then, they decide which machine is

best for their needs. Rather than choosing directly, people attempt to gain information prior to purchasing a computer. EL resembles people wanting to buy a new computer.

In the proposed EnLeM approach, with the training dataset (i.e., labeled URLs), three classifiers are created, and each individual classifier provides its prediction label on the test dataset (i.e., unlabeled URLs). Then, every classifier provides a label, i.e., phishing (1) or legitimate (0). However, this step functions to identify the label that provides the most classifiers and make the final decision.

In the rare case of a complete tie (1–1–1), the final decision defaults to the prediction of the Random Forest classifier, since it achieved the highest standalone accuracy among the base learners in our benchmarking. This rule ensures consistent and reproducible outcomes when no clear majority exists.

3.5.3 Proposed algorithm

As shown in Algorithm 1, we initialize a feature set comprising 30 features ($f_1, f_2, \dots, f_{30}$) for both the training and testing datasets. First, the dataset includes 6157 labeled legitimate webpages and 4898 labeled phishing webpages (line 1), which ensures a comprehensive dataset to train and evaluate the proposed model. In addition, we eliminate a subset of irrelevant features or attributes (Sect. 3.3). To construct the proposed model, three base classifiers, i.e., the *decision tree*, *RF*, and *k-nearest neighbor* classifiers, are constructed using the labeled URLs (e.g., L_{web}). Theoretically, we could use any number of constituent classifiers (i.e., greater than three) in the EL approach. However, here, we use only three as a proof of concept.

In addition, the classifier predicts the labels of the unlabeled URLs (e.g., U_{web}), as shown in lines 6–15 in Algorithm 1. Finally, a label is a voting base designed by

aggregating each class's label (phishing or nonphishing). In Algorithm 1, the label returned for U_{web} is associated with the maximum votes that provide the three different classifiers.

Here, we consider the case for the webpage $U = \text{"https://fmovies.ps"} (Disclaimer: This is a real-world case study, and readers should not interact with this website). The website has 30 distinct attributes are shown in Table 3.$

For this case, assume that the *decision tree* classifier classifies this webpage as phishing, the *RF* classifier classifies it as a legitimate webpage, and the *k-nearest neighbor* classifier determines it is a phishing webpage. Consequently, the proposed EnLeM will predict that it is a phishing webpage according to Algorithm 1.

Table 3 Attributes of an example phishing webpage

Attribute	Values
Having IP address	−1
URL length	1
Shortening service	1
Having 'at' symbol	1
Double slash redirecting	−1
Prefix suffix	−1
Having sub domain	−1
SSLfinal state	−1
Domain registration length	−1
Favicon	1
Port	1
HTTPS token	−1
Request URL	1
URL of anchor	−1
Links in tags	1
SFH (server form handler)	−1
Submitting to e-mail	−1
Abnormal URL	−1
Redirect page	0
On mouse over (using to hide link)	1
Right click	1
Using pop-up window	1
Iframe	1
Age of domain	−1
DNS record	−1
Web traffic	−1
PageRank	−1
Google Index	1
Links pointing to page	1
Statistical report	−1

Algorithm 1 Proposed learning approach

Input: L_{web} : Set of labeled training data, U_{web} : Set of unlabeled testing data from the UCI Phishing websites dataset.

Output: 1 if a website is phishing; 0 if a website is legitimate.

```

// Define feature sets for labeled and unlabeled data
1 Let  $L_{features}$  be the feature set of  $L_{web}$ 
2 Let  $U_{features}$  be the feature set of  $U_{web}$ 
// Number of classifiers in the ensemble
3 Let  $N = 3$ 
// Initialize an empty list of classifiers
4 Initialize an empty list  $EnsembleClassifiers$ 
// Train classifiers
5  $Classifier_1 = \text{Decision\_Tree}(L_{features})$ 
6  $Classifier_2 = \text{Random\_Forest}(L_{features})$ 
7  $Classifier_3 = \text{K\_Nearest\_Neighbors}(L_{features})$ 
8  $EnsembleClassifiers.append(Classifier_1)$ 
9  $EnsembleClassifiers.append(Classifier_2)$ 
10  $EnsembleClassifiers.append(Classifier_3)$ 
11 foreach  $F$  in  $U_{web}$  do
    // Initialize counters for phishing and legitimate predictions
    12 Let  $X_P(\text{Phishing}) = 0$  and  $X_L(\text{Legitimate}) = 0$ 
    13 foreach  $Classifier_j$  in  $EnsembleClassifiers$  do
        // Make predictions using each classifier
        14  $Predicted\_Label\_for\_F_j = Classifier_j(F_{features})$ 
        // Count the votes for each class
        15  $X_P^j(\text{Phishing}) = 1$  if  $Predicted\_Label\_for\_F_j$  is phishing, else 0
        16  $X_L^j(\text{Legitimate}) = 1$  if  $Predicted\_Label\_for\_F_j$  is legitimate, else 0
        // Update the counters
        17  $X_P(\text{Phishing}) = X_P(\text{Phishing}) + X_P^j(\text{Phishing})$ 
        18  $X_L(\text{Legitimate}) = X_L(\text{Legitimate}) + X_L^j(\text{Legitimate})$ 
    // Assign the final label based on majority voting
    19 if  $X_P(\text{Phishing}) \geq X_L(\text{Legitimate})$  then
        20 Return: 1 // Website is predicted as phishing
    21 else
        22 Return: 0 // Website is predicted as legitimate

```

3.5.4 Base classifiers

In the context of the proposed model for phishing website detection, we combine three supervised learning methods due to their several advantages. First, there are many known outcomes (i.e., phishing or legitimate) in our dataset, which are crucial for training the model to recognize patterns and make predictions. Second, the combination of three supervised models can predict outcomes accurately based on the familiar patterns present in the training data. By learning from labeled examples, they can generalize their understanding and make informed predictions on new but similar instances. Finally, they are more efficient and adjust their parameters to realize better optimization performance compared to unsupervised learning models. In addition, the combination of these techniques provides a more comprehensive solution when attempting to detect new phishing websites.

In the following, we summarize the three constituent base classifiers (i.e., the DT, RF, and *k*-NN) in the proposed EnLeM.

(i) *Decision tree* A DT [37] is a flowchart constructed using an algorithmic approach that identifies the ways to split a dataset based on different conditions. With this method, it is easy to handle categorical and continuous data, the model is easy to read and interpret, and the generated rules are human-understandable. However, the DT method is unstable and incurs high computational costs.

(ii) *Random forest* Bagging (bootstrap aggregation) is an intuitive and straightforward ensemble-based technique that provides surprisingly good performance [38]. In this approach, multiple instances of the base classifier model are used for classification. First, bagging creates multiple bootstrap samples from the original dataset, where each bootstrap sample works as a new independent dataset. The individual classifiers are then trained on the bootstrapped samples. Finally, the outputs of the individual models are combined by averaging or voting to create a single output with minor variance.

The RF [39, 40] is an ensemble of DTs that employs a greedy method to obtain the best split. This technique reduces the variance and overfitting of the model, and it provides information about significant features from the dataset. However, it is difficult for humans to interrupt, and this method exhibits poor performance on imbalanced data.

(iii) *k-nearest neighbor* The k -NN [41, 42] technique selects the k -closest examples from the dataset. It is both intuitive and straightforward to understand, and new data can be added seamlessly. However, this method does not work effectively on large datasets with high dimensions, and it requires feature scaling.

The information on the important hyper-parameters set for the three base learners are given in Table 4.

The hyperparameters summarized in Table 4 were not set manually; they were optimized using grid search with 10-fold cross-validation on the training set. For each base learner, we explored a defined search space (e.g., maximum depth {3, 5, 7, 10} for decision tree and random forest; number of estimators {50, 100, 200} for random forest; $k \in \{3, 5, 7, 9\}$ for k -NN with Euclidean and Manhattan distance metrics). The final values reported in Table 4 correspond to those that yielded the highest average validation accuracy across the 10 folds. This systematic procedure ensured fairness and reproducibility compared to relying on default or manually chosen settings.

3.6 Benchmarking

Motivated by previous studies, we compared the proposed EnLeM to five ML models, i.e., the DT, RF, AdaBoost, SVM, and k -NN models, and two DL-based models, i.e., the CNN and the LSTM models. If the ML-based and DL-based classifiers individually predict the labels as phishing or legitimate, then each classifier is referred to as a single-view learning model. However, individual learning approaches can only make up some types of applications. In contrast, multiple classifiers

Table 4 Hyper-parameters of base learners

Hyper-parameters	Values
Decision tree	
Maximum depth	5
Minimum samples before split	2
Minimum samples in leaf	1
Criterion	gini
Random forest	
No. of estimators	100
Maximum no. of features	sqrt
Maximum depth	5
Minimum samples before split	2
Minimum samples in leaf	1
Bootstrap	True
k-nearest neighbor	
No. of neighbors (k)	5
Distance metric	minkowski

produce multiple decisions and can perform class predictions on unlabeled data based on consensus.

Bagging-based EL allows more than two weak models, aggregates their predictions, and provides excellent accuracy; however, this method frequently results in high bias and underfitting when not modeled correctly [43]. In addition, boosting-based EL is challenging because of its high complexity.

3.6.1 Tradition machine learning methods

A comprehensive list of machine learning (ML) approaches was created and used to predict phishing e-mails or websites [44, 45]. However, the outcomes of these ML approaches are dependent on the characteristics of the dataset. Thus, no single model performs the best for different target problems [46].

For example, Yadav et al. [47] employed DT (J48), RF, and logistic regression with about 99% precision with the RF algorithm best differentiates between phishing and legitimate (i.e., non-spam) e-mails.

In another work, Kolla et al. [48] employed several ML algorithms (e.g., DTs, RFs, multilayer perceptions, SVMs, and XGBoost) to predict phishing URLs. In that study, the random forest algorithm obtained the highest accuracy of 90%. Based on phishing detection, for instance, in this paper[49], they only trained the DT model and obtained approximately 90% accuracy, SVM with 95.66% accuracy in [50]. However, Ojewumi et al. employed three ML algorithms and obtained 93.39% accuracy with the k -NN technique, 91.74% accuracy with the SVM method, and 98.35% accuracy with the RF technique [51]. Collectively, these findings represent

variability in terms of the performance metric values obtained across several studies.

Thus, in the current study, we experimented with the aforementioned ML algorithms, namely, (i) DT, (ii) RF, (iii) AdaBoost, (iv) SVM, and (v) k -NN, which are frequently used for the phishing website detection task. The underlying concepts, advantages, and disadvantages of these ML algorithms are summarized in the following sections.

Here, note that the DT, RF, and k -NN methods were described previously in Sect. 3.5.4. The descriptions of AdaBoost and SVM are given below.

AdaBoost Boosting [52] is a meta-algorithm that can be used to realize model averaging. Bagging is primarily used for classification problems and occasionally for regression problems. Boosting creates weak classifiers, and the sequence of models is created iteratively. Here, each model is trained on a complete dataset, and the boosting process attempts to add higher weights to the instances that were misclassified by the previous model. By taking into account the accuracy score and the weight of the output of each model, all sequences of the models are combined using voting (for classification) or averaging (for regression) to create the final estimation.

AdaBoost [53] is an ensemble of DTs with only a single split (one level). It is easy to program quickly, and it can be combined with other models flexibly. However, the base classifier is very weak and vulnerable to uniform noise.

SVM An SVM [41] selects the best hyperplane that maximizes the separability between classes. Note that SVM methods are robust against overfitting, and the model can solve any complex problem with a perfect kernel function. However, SVMs are memory-intensive and require a longer training time than other models, such as DT and NB.

3.6.2 Deep learning methods

In addition to the traditional ML methods, the use of DL methods is becoming increasingly prominent for various tasks. In the following, we briefly describe two DL methods that were used in this study for benchmarking, i.e., (i) 1D-CNN and (ii) LSTM.

Convolutional neural network Artificial neural networks (ANN) are actively being developed in a wide range of fields. CNNs evolved from ANNs, and the first CNN to be similar to contemporary network structures was the Neocognitron network [54]. In addition, the first real CNN, referred to as the LeNet 5 network, has been

widely used [55] to recognize handwritten numbers. The basic architecture of a CNN classifier is described in the literature [56].

The CNN model can also utilize a one-dimensional structure (i.e., the 1D-CNN) for a processed dataset [20]. Note that a particular segment may derive an informative feature from the overall dataset so that 1D-CNN models work efficiently. Some significant applications of 1D-CNNs include automatic speech recognition, real-time electrocardiogram monitoring, vibration-based structural damage detection in civil infrastructure, and high-power multilevel converters [57]. In this study, we used a 1D-CNN model containing a convolutional layer, a pooling layer, and one fully connected layer with an activation function. We classified the UCI dataset into two classes (phishing or legitimate) using the 1D-CNN. In addition, we used the ReLu activation function in the CNN model to mitigate the vanishing and exploding gradient problems. We also used another classical nonlinear activation function, i.e., the sigmoid activation function, which is more efficient when performing training on a massive amount of data.

Long short-term memory The LSTM is a type of recurrent neural network used in DL that was proposed by Hochreiter et al. [58] between 1995 and 1997 to handle the vanishing gradient problem when introducing constant error carousel units. Unlike standard feedforward neural networks, the LSTM method has feedback connections. Some applications of this technique include language modeling, handwriting recognition, speech recognition, image processing, music generation, and anomaly detection. LSTM has several units that are similar to cells, an input gate, an output gate, and a forget gate. Additional details about the basic architecture of the LSTM model can be found in the literature [59]. Although LSTM is primarily used for sequential data, it can also be applied in classifying non-sequential tabular data [60].

4 Experimental evaluation

4.1 Experimental design

In our experiments, we compared various ML and DL models to leverage their distinct capabilities. In this study, the scikit-learn library, which is known for its ease of use and versatility, provided a solid foundation for implementing traditional ML models. In addition, Keras, an open-source library that provides a Python interface for ANNs that integrates seamlessly with TensorFlow, offered a high-level interface for constructing and

training neural networks, while Numpy (an open-source library for numerical data processing) and Pandas (an open-source library for handling data frames) facilitated efficient data manipulation and preprocessing.

The choice of the ML and DL models involved a comprehensive evaluation of their applicability to our target problem. For example, DTs were selected for their interpretability, and they provide valuable insights into feature importance. The ensemble methods (i.e., the RF and AdaBoost methods) were selected to mitigate overfitting and enhance predictive performance by aggregating multiple weak learners. In addition, SVMs were selected for their proficiency in high-dimensional spaces, and the k-NN method offers a localized approach that is well-suited for capturing intricate patterns. In terms of the DL methods, the 1D-CNN and LSTM methods were selected for their aptitude in terms of extracting hierarchical features from sequential data. Specifically, we used the LSTM model because it can capture temporal dependencies in the data (i.e., the sequential nature of web traffic data). The binary cross-entropy function [61], which is given in Eq. 4, was selected as the loss function in our approach. To minimize this loss while training the CNN and LSTM models, we employed the gradient-based Adam optimizer [62].

$$Loss = -\frac{1}{N} \sum_{i=1}^N y_i \cdot \log y'_i + (1 - y_i) \cdot \log(1 - y'_i) \quad (4)$$

Here, y_i acts as the desired output, and y'_i is the predicted output of the i^{th} data.

All training was performed using the default settings of the defined libraries for the ML models, and an initial learning rate of 0.001 and minibatch size of 100 were used for the DL models. The computational resources, comprising an NVIDIA Tesla K80 GPU (12GB) and an Xeon hyperthreaded processor (2.3GHz) on Google Colab, played a pivotal role in our experimental setup. However, it is imperative to acknowledge the potential limitations associated with this selection of computation resources, e.g., constraints on both memory and processing power.

4.2 Evaluation metrics

After cross-validating the proposed approaches, we evaluated the performance of the proposed model. In this study, we used a set of evaluation metrics that are commonly used to assess the performance of various classification problems: precision, recall, the F1-score, and accuracy [63]. These performance metrics are given in the equations below.

Let true positive (TP) indicate the number of phishing websites that are correctly classified as phishing, true negative (TN) the number of legitimate websites that

are correctly classified as legitimate, false positive (FP) the number of legitimate websites that are incorrectly classified as phishing, and false negative (FN) the number of phishing websites that are incorrectly classified as legitimate.

Precision is defined as the number of TP divided by the sum of TP and FP .

$$\text{Precision} = \frac{TP}{TP + FP} \quad (5)$$

Recall is defined as the number of TP divided by the sum of TP and FN .

$$\text{Recall} = \frac{TP}{TP + FN} \quad (6)$$

The F1-score is the harmonic mean of precision and recall.

$$\text{F1-score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \quad (7)$$

Finally, accuracy is defined as the number of correct predictions divided by the total number of predictions.

$$\text{Accuracy} = \frac{TP + TN}{TP + FP + TN + FN} \quad (8)$$

4.3 Experimental results and discussion

A comparative experiment was conducted to determine the performance of each classification model. Here, we used most common cross-validation technique, i.e., k -fold cross-validation (where $k = 10$ was used in the experiment). Cross-validation on a k -fold scale divides the dataset into k -folds, with each fold serving as a performance test for the model. To complete this operation, k repetitions of the training and testing data are required. We divided the data into 10 folds using k -folds cross-validation, and each fold was roughly identical to the other folds in the dataset. Each iteration of the trained learning scheme was repeated nine times, and the performance of the learning approaches was evaluated using the remaining fold, i.e., the test set. The learning scheme was repeated 10 times on the UCI phishing dataset, and the prediction accuracies of the 10 repetitions were averaged to obtain the overall prediction results in terms of precision, recall, F1-score, and accuracy.

4.3.1 Performance without feature selection

To evaluate the performance of the proposed EnLeM compared to the ML and DL classifiers, we initially assessed all

30 features prior to implementing feature selection techniques. The results in Table 6 include the weighted average precision, recall, F1-score, and accuracy values for all compared algorithms, including the respective execution times. The compared models were evaluated in terms of accuracy, precision, recall, and F1 score because these are fundamental indicators of a classifier's efficacy, and they provide insights into a model's ability to classify instances correctly and avoid FP or negatives. We also considered the execution times because this information sheds light on the computational efficiency of each algorithm. On the experimental dataset, the 1D CNN model obtained comparatively good results, and the proposed model outperformed all compared models in terms of accuracy, precision, recall, and F1-score metrics. The accuracy, precision, recall, and F1-score results obtained by the 1D-CNN model were 96.75%, 96.53%, 96.24%, and 96.37%, respectively. The proposed model obtained superior results with corresponding values of 97.21%, 97.06%, 97.97%, and 97.51%. In addition, the DT, RF, Adaboost, k-NN, SVM, and LSTM models obtained accuracy values of 96.75%, 92.37%, 93.21%, 93.21%, 91.71%, 94.74%, 92.79%, and 90.84%, respectively.

The accuracy results obtained by the compared models without feature selection are shown in Fig. 4.

The F1-score is very important for the phishing website detection task on the experimental dataset because it includes more legitimate websites (6157) than phishing web pages (4898). As mentioned previously, the F1-score is the harmonic mean of precision and recall, which provides a balanced measure of FP and FN errors to find a reasonable tradeoff. The F1-score of the proposed model was 97.51%, and the 1D-CNN, KNN, RF, SVM, DT, AdaBoost, and LSTM classifiers followed in descending order: 96.24%, 95.47%, 94.04%, 93.60%, 93.42%, 92.58%, and 91.75%, respectively. Figure 5 compares the F1-score values obtained by the compared different classifiers without feature selection.

4.3.2 Performance with feature selection

In this section, we discuss the results obtained with the univariate feature selection technique, which leverages MI to identify the top 20 important features for classifying phishing websites. This method was selected for its effectiveness in selecting relevant features for the target task. We compared the performance of seven ML and two DL algorithms, all of which utilized the same top 20 features, and we observed notable variations in terms of the precision, recall, F1-score, and accuracy results, as shown in Table 6. We found that the proposed model outperformed the compared models, with an excellent accuracy value of 96.26%. Most of the trained models, e.g., the k-NN, DT, RF, AdaBoost, SVM, and LSTM models, demonstrated accuracy values ranging from 91.26 to 94.35%. However, the 1D-CNN model obtained a comparatively lower accuracy of 78.06% because it typically operates on multivariate data, and applying them directly to univariate feature selection may not exploit its capabilities fully. Similar observations were made by other researchers regarding the poor performance of deep learning such as CNN in tabular data classification [64]. Figure 6 compares the accuracy of several classifiers with the feature selection technique.

We found that the proposed model outperformed the individual algorithms and obtained the maximum precision, recall, and F1-score values of 96.36%, 97.96%, and 96.66%, respectively. These results suggest that the proposed method excels in terms of both accuracy and capturing a significant proportion of TPs, which is crucial for the phishing detection task. In contrast, the 1D-CNN model was the least accurate classifier, obtaining only 78.06% accuracy and a recall value of 97.66%. These results prove that the 1D-CNN model faced challenges in achieving high accuracy despite its recall performance, which is potentially due to the use of fewer features during training. The proposed model showcased a balanced

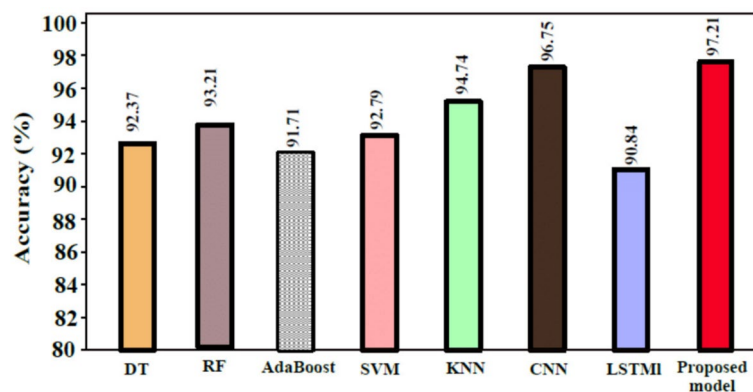


Fig. 4 Accuracy of different classifiers without feature selection

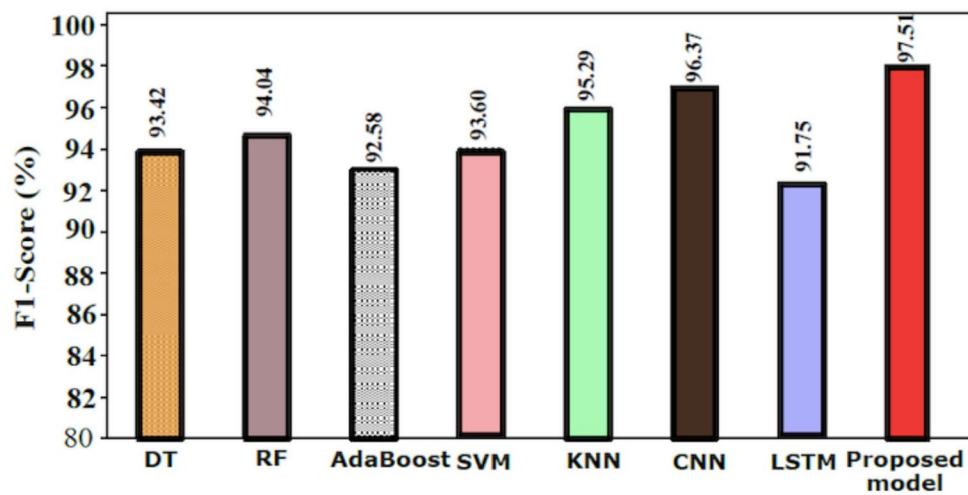


Fig. 5 F1-scores of different classifiers without feature selection

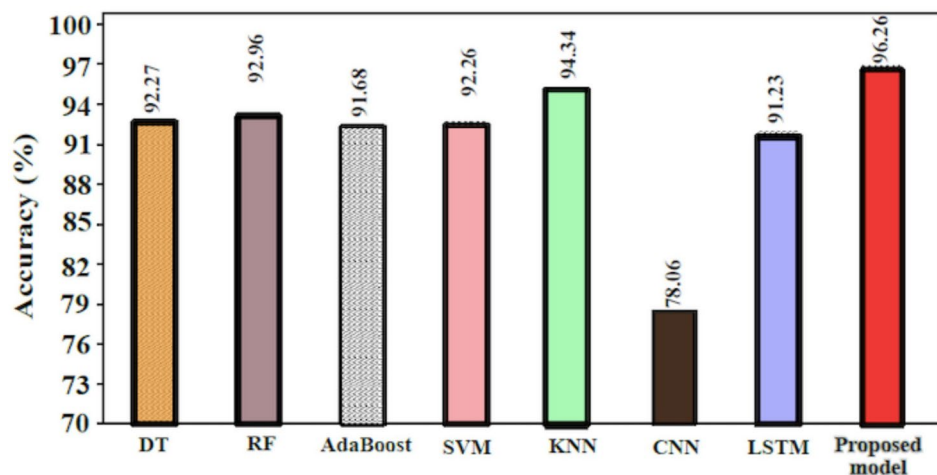


Fig. 6 Accuracy of different classifiers with feature selection

F1-score (96.06%), and the remaining individual classifiers obtained F1-scores ranging from 91.47 to 94.95%. In contrast, the 1D-CNN model obtained the lowest F1-score. Figure 7 compares the F1-score results of the compared models with feature selection.

4.3.3 Ablation on feature selection robustness

To further evaluate the robustness of the Mutual Information (MI)-based SelectKBest feature selection, we performed an ablation study by varying the number of selected features $k \in \{10, 15, 20, 25, 30\}$. The results (Table 5) show that the performance of EnLeM remains stable across different thresholds, with accuracy and F1-score variations within $\pm 0.7\%$. Importantly, reducing k decreases execution time while preserving detection

performance, demonstrating that SelectKBest is not overly sensitive to noisy or redundant features.

4.3.4 Comparison with and without feature selection

Table 6 compares the accuracy of the classifiers obtained with and without the univariate feature selection technique. As can be seen, the accuracy of nearly all algorithms remained constant or statistically insignificant without and with the feature selection, except for the 1D-CNN model.

These results confirm that the univariate feature selection method is successful in determining important features, which facilitates the accurate classification of phishing websites.

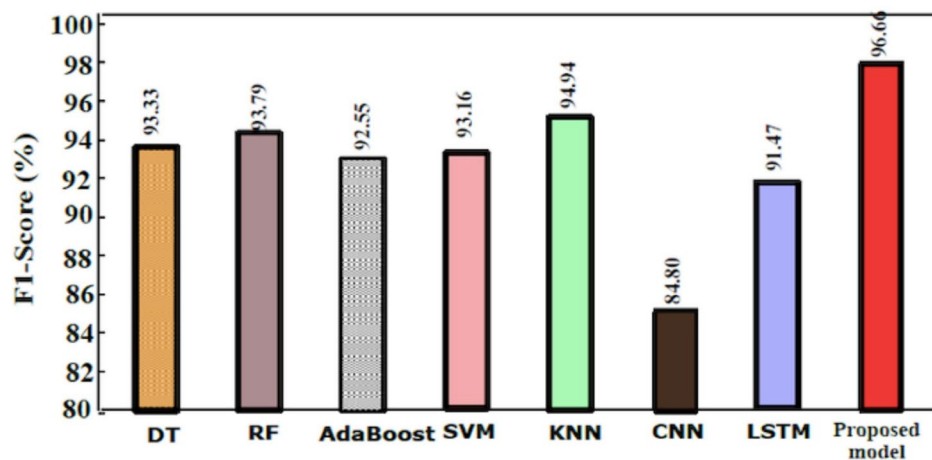


Fig. 7 F1-scores of different classifiers with feature selection

Table 5 Ablation study on number of selected features (k) using MI-based SelectKBest

Top- k features	Accuracy (%)	F1-score (%)	Execution time (s)
10	96.42	96.10	7.81
15	96.73	96.38	8.12
20	97.21	97.51	8.72
25	96.98	97.30	9.21
30 (all)	97.05	97.28	10.01

Figure 8 shows the execution times of the classification models, including a comparison of the execution times with and without the feature selection technique. Generally, we found that feature selection helps reduce the execution time. Figure 8 also shows how feature selection

reduces the time required to execute the models. As simple learners, the DT models have faster execution times than the other techniques, whereas the LSTM model requires more time for training due to memory bandwidth-constrained computing.

For the proposed EnLeM method, although the performance metrics obtained with the feature selection technique were slightly lower than those obtained without feature selection, the differences are negligible. The results of a paired t -test [65] conducted to assess the four performance metrics (accuracy, precision, recall, and F1-score) with and without feature selection gave a p -value of 0.1353, which is higher than the threshold of 0.05 and thus not statistically significant. However, in terms of the execution time, the difference was 1.29s (representing a roughly 13% increase in execution time).

Table 6 Comparison of performance metrics obtained by ML models, DL-based methods, and the proposed EnLeM (the highest value for each metric, except execution time, is shown in bold)

Classifiers	Before feature selection					After feature selection				
	Accuracy	Precision	Recall	F1-score	Ex. time (s)	Accuracy	Precision	Recall	F1-score	Ex. time (s)
Decision tree (DT)	92.37	90.03	97.09	93.42	0.31	92.27	89.82	97.14	93.33	0.27
Random forest (RF)	93.21	92.04	96.13	94.04	4.70	92.96	92.28	96.13	93.79	4.63
AdaBoost	91.71	92.37	92.80	92.58	0.63	91.68	92.31	92.81	92.55	0.56
Support vector machine (SVM)	92.79	92.56	94.67	93.60	27.03	92.26	91.72	94.64	93.16	18.84
k -Nearest neighbor (k -NN)	94.74	95.12	95.47	95.29	7.18	94.34	94.56	95.32	94.94	4.54
1D-convolutional neural network (1D-CNN)	96.75	96.53	96.24	96.37	32.02	78.06	76.93	97.66	84.80	20.04
Long short-term memory (LSTM)	90.84	91.90	91.85	91.75	267.77	91.23	90.69	92.40	91.47	172.02
Proposed approach (EnLeM)	97.21	97.06	97.97	97.51	10.01	96.26	96.36	97.96	96.66	8.72

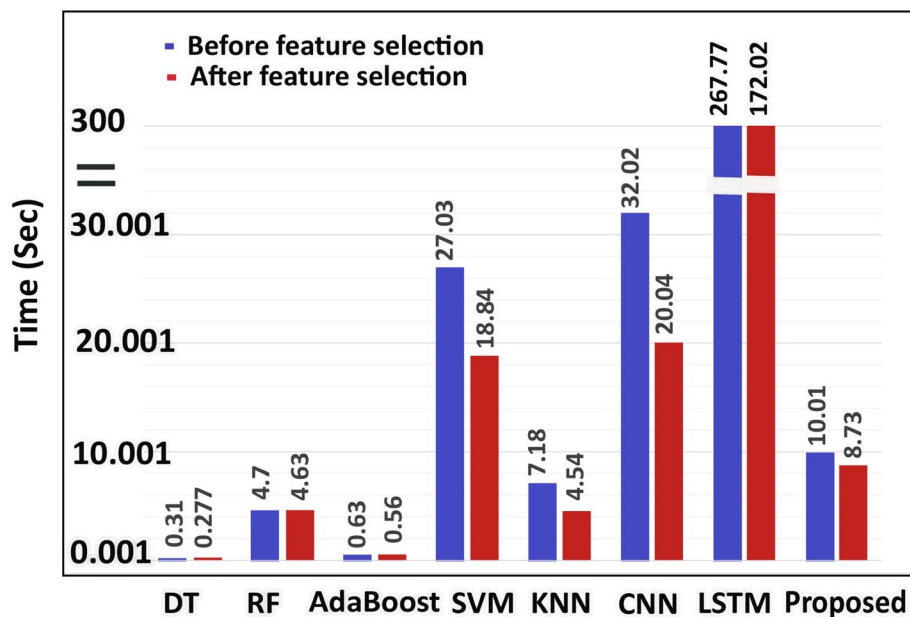


Fig. 8 Comparison of execution times with and without feature selection

This can be useful in mission-critical applications with high traffic volume and high throughput.

In summary, the experimental results indicate that the proposed EnLeM method is an effective technique to predict phishing websites with the highest accuracy, precision, recall, and F1-score results with a relatively short execution time.

5 Conclusion and future work

This work presented *EnLeM*, a lightweight ensemble for phishing detection that balances accuracy, efficiency, and interpretability. Embedded MI-based feature selection within CV prevents data leakage and improves runtime without degrading accuracy. On the UCI dataset, EnLeM achieved 97.21% accuracy and 97.51% F1, outperforming ML baselines and offering higher efficiency than CNN/LSTM. Future work will explore integration with federated or split learning for privacy-preserving deployment and investigate advanced dimensionality reduction (e.g., autoencoders) and evaluation on larger, more diverse datasets.

Acknowledgements

The authors thank the Coastal Virginia node of the Commonwealth Cyber Initiative (COVA CCI), USA for their valuable support.

Authors' contributions

All authors conceived the research idea. MNY and MARR implemented the method and conducted the experiments. BCS, ZAlom, ZAung, and MA supervised the research process and carried out the analysis of results. MNY and MARR wrote the initial draft. BCS, ZAlom, ZAung, and MA wrote the final draft. All authors reviewed the manuscript.

Funding

This work is funded by Khalifa University of Science and Technology, Abu Dhabi, UAE. The authors, therefore, gratefully acknowledge the financial support.

Data availability

The dataset is available at the UCI Machine Learning Repository (<https://doi.org/10.24432/C51W2X>).

Declarations

Competing interests

The authors declare that they have no competing interests.

Received: 20 May 2025 Accepted: 29 October 2025

Published online: 05 December 2025

References

1. P. Stavroulakis, M. Stamp, *Handbook of information and communication security* (Springer Science & Business Media, Heidelberg, 2010)
2. I.K. Acheampong. The state of phishing attack vector (2024). Slides at [https://github.com/OWASP/www-chapter-ghana/blob/master/assets/slides/Phishing_Presentation\(OWASP_Ghana\).pdf](https://github.com/OWASP/www-chapter-ghana/blob/master/assets/slides/Phishing_Presentation(OWASP_Ghana).pdf). Accessed 7 Nov 2025
3. Retruster. (2020). Blog at <https://retruster.com>. Accessed 7 Nov 2025
4. Los angeles times. (2020). Blog at <https://www.latimes.com/business/la-fi-mh-anthem-is-warning-consumers-20150306-column.html>. Accessed 7 Nov 2025
5. L.A.T. Nguyen, B.L. To, H.K. Nguyen, M.H. Nguyen, in *2014 international conference on computing, management and telecommunications (Com-ManTel)*. A novel approach for phishing detection using url-based heuristic (IEEE, 2014), pp. 298–303
6. T. Suleman, A survey on web phishing detection techniques. *International Journal for Electronic Crime Investigation* **5**(2), 25–36 (2021)
7. S. Revathy, S. Sathya Priya, S. Rafi, P. Pranideep, D. Rajesh, in *Ambient Communications and Computer Systems: Proceedings of RACCCS 2021*. Detection of active attacks using ensemble machine learning approach (Springer, Singapore, 2022), pp. 507–519

8. R. Dhamija, J.D. Tygar, M. Hearst, in *Proceedings of the SIGCHI conference on Human Factors in computing systems*. Why phishing works (ACM, New York, 2006), pp. 581–590
9. W. Wei, Q. Ke, J. Nowak, M. Korytkowski, R. Scherer, M. Woźniak, Accuracy and fast url phishing detector: a convolutional neural network approach. *Comput. Netw.* **178**, 107275 (2020)
10. J. Mao, J. Bian, W. Tian, S. Zhu, T. Wei, A. Li, Z. Liang, Phishing page detection via learning classifiers from page layout feature. *EURASIP J. Wirel. Commun. Netw.* **2019**(1), 1–14 (2019)
11. Y. Masoudi-Sobhanzadeh, H. Motieghader, A. Masoudi-Nejad, Feature-select: a software for feature selection based on machine learning approaches. *BMC Bioinformatics* **20**, 1–17 (2019)
12. K. Shaukat, S. Luo, V. Varadarajan, I.A. Hameed, S. Chen, D. Liu, J. Li, Performance comparison and current challenges of using machine learning techniques in cybersecurity. *Energies* **13**(10), 2509 (2020)
13. D. Müller, I. Soto-Rey, F. Kramer, An analysis on ensemble learning optimized medical image classification with deep convolutional neural networks. *IEEE Access* **10**, 66467–66480 (2022)
14. G. Rao, C. Peng, L. Zhang, X. Wang, Z. Feng, in *International Conference on Knowledge Science, Engineering and Management*. A knowledge enhanced ensemble learning model for mental disorder detection on social media (Springer, Cham, Switzerland, 2020), pp. 181–192
15. Y. Mao, S. Liu, D. Gong, A text mining and ensemble learning based approach for credit risk prediction. *Tehnički Vjesn* **30**(1), 138–147 (2023)
16. Z. Abou El Houda, B. Brik, L. Khoukhi, in *2022 IEEE 47th Conference on Local Computer Networks (LCN)*. Ensemble learning for intrusion detection in sdn-based zero touch smart grid systems (IEEE, Edmonton, 2022), pp. 149–156
17. A.A. Ubung, S.K.B. Jasmi, A. Abdullah, N. Jhanjhi, M. Supramaniam, Phishing website detection: An improved accuracy through feature selection and ensemble learning. *Int. J. Adv. Comput. Sci. Appl.* **10**(1), (2019), pp. 252–257
18. F. Toolan, J. Carthy, in *2009 eCrime researchers summit*. Phishing detection using classifier ensembles (IEEE, Tacoma, 2009), pp. 1–9
19. I. Salihovic, H. Serdarevic, J. Kevric, in *International Symposium on Innovative and Interdisciplinary Applications of Advanced Technologies*. The role of feature selection in machine learning for detection of spam and phishing attacks (Springer, Cham, Switzerland, 2018), pp. 476–483
20. S.Y. Yerima, M.K. Alzaylaee, in *2020 3rd International Conference on Computer Applications & Information Security (ICCAIS)*. High accuracy phishing detection based on convolutional neural networks (IEEE, Riyadh, 2020), pp. 1–6
21. H. Yuan, X. Chen, Y. Li, Z. Yang, W. Liu, in *2018 24th International Conference on Pattern Recognition (ICPR)*. Detecting phishing websites and targets based on urls and webpage links (IEEE, Beijing, 2018), pp. 3669–3674
22. A.K. Jain, B.B. Gupta, Towards detection of phishing websites on client-side using machine learning based approach. *Telecommun. Syst.* **68**(4), 687–700 (2018)
23. X. Gu, H. Wang, T. Ni, An efficient approach to detecting phishing web. *J. Comput. Inf. Syst.* **9**(14), 5553–5560 (2013)
24. M. Moghimi, A.Y. Varjani, New rule-based phishing detection method. *Expert Syst. Appl.* **53**, 231–242 (2016)
25. N. Zhang, Y. Yuan, Phishing detection using neural network. CS229 lecture notes (Stanford University, California, 2012). <https://cs229.stanford.edu/proj2012/ZhangYuan-PhishingDetectionUsingNeuralNetwork.pdf>
26. R.S. Rao, A.R. Pais, Detection of phishing websites using an efficient feature-based machine learning framework. *Neural Comput. Appl.* **31**(8), 3851–3873 (2019)
27. D. Aksu, Z. Turgut, S. Üstebay, M.A. Aydin, in *International Telecommunications Conference*. Phishing analysis of websites using classification techniques (Springer, Singapore, 2019), pp. 251–258
28. D. Zhang, Z. Yan, H. Jiang, T. Kim, A domain-feature enhanced classification model for the detection of Chinese phishing e-business websites. *Inf. Manag.* **51**(7), 845–853 (2014)
29. A.A. Khan, A.A. Laghari, A.M. Baqasah, R. Bacarra, R. Alroobaea, M. Alsafyani, J.A.J. Alsayaydeh, Bdlit-iomt—a novel architecture: Svm machine learning for robust and secure data processing in internet of medical things with blockchain cybersecurity. *J. Supercomput.* **81**(1), 271 (2025)
30. C. Whittaker, B. Ryner, M. Nazif, in *Proceedings of the Network and Distributed System Security Symposium, NDSS 2010*. Large-scale automatic classification of phishing pages (Internet Society, San Diego, 2010), pp. 1–14.
31. R. Mohammad, L. McCluskey. Phishing Websites. UCI Machine Learning Repository (2015). <https://doi.org/10.24432/C51W2X>
32. R.M. Mohammad, F. Thabtah, L. McCluskey, *Phishing websites features* (School of Computing and Engineering, University of Huddersfield, UK, 2015)
33. S. García, J. Luengo, F. Herrera, *Data preprocessing in data mining*, vol. 72 (Springer, Cham, 2015)
34. G. Dougherty, *Pattern recognition and classification: an introduction* (Springer Science & Business Media, New York, 2012)
35. R. Polikar, Ensemble learning. *Scholarpedia* **4**(1), 2776 (2009). <https://doi.org/10.4249/scholarpedia.2776>
36. B.V. Dasarathy, B.V. Sheela, A composite classifier system design: concepts and methodology. *Proc. IEEE* **67**(5), 708–713 (1979)
37. D.N. Kumar, N.S.R. Hemanth, S. Premnath, V.N. Kumar, S. Uma, Detection of phishing websites using an efficient machine learning framework. *Int. J. Eng. Res. Technol.* **9**(5), 1282–1286 (2020)
38. L. Breiman, Bagging predictors. *Mach. Learn.* **24**(2), 123–140 (1996)
39. A. Subasi, E. Molah, F. Almkallawi, T.J. Chaudhery, in *2017 International conference on electrical and computing technologies and applications (ICECTA)*. Intelligent phishing website detection using random forest classifier (IEEE, Ras Al Khaimah, 2017), pp. 1–5
40. R. Vallepu, M. Karunakaran, in *AIP Conference Proceedings*. An innovative method to improve performance analysis in classification with accuracy of phishing websites using random forest algorithm by comparing with support vector machine algorithm, vol. 2655 (International Conference on Environmental Engineering and Material Sciences: ICEEMS-2021, Coimbatore, 2023)
41. A. Altaher, Phishing websites classification using hybrid svm and knn approach. *Int. J. Adv. Comput. Sci. Appl.* **8**(6) (2017), pp. 90–95
42. M. Tavallae, E. Bagheri, W. Lu, A.A. Ghorbani, in *2009 IEEE symposium on computational intelligence for security and defense applications*. A detailed analysis of the KDD Cup 99 data set (IEEE, Ottawa, 2009), pp. 1–6
43. Techniques of ensemble learning: Bagging vs boosting | educba. (2023). <https://www.educba.com/bagging-and-boosting/>. Accessed 07 Apr 2024
44. V. Shahrivari, M.M. Darabi, M. Izadi, Phishing detection using machine learning techniques. (2020). arXiv preprint arXiv:2009.11116. <https://arxiv.org/abs/2009.11116>
45. S. Abu-Nimeh, D. Nappa, X. Wang, S. Nair, in *Proceedings of the anti-phishing working groups 2nd annual eCrime researchers summit*. A comparison of machine learning techniques for phishing detection (ACM, New York, 2007), pp. 60–69
46. M. Fernández-Delgado, E. Cernadas, S. Barro, D. Amorim, Do we need hundreds of classifiers to solve real world classification problems? *J. Mach. Learn. Res.* **15**(1), 3133–3181 (2014)
47. N. Yadav, S.P. Panda, in *International Conference on Innovative Computing and Communications*. Feature selection for email phishing detection using machine learning (Springer, Singapore, 2022), pp. 365–378
48. J. Kolla, S. Praneeth, M.S. Baig, G. Karri, A comparison study of machine learning techniques for phishing detection. *Journal of Business and Information Systems (e-ISSN: 2685-2543)* **4**(1), 21–33 (2022)
49. D. Pathak, M. Ammar, M. Bhandari, Phishing detection approach using machine learning. *Int. Res. J. Modernization Eng. Technol. Sci.* **4**(5), 4233–1238 (2022)
50. M. Mohammed, K.K. Prasanth, S.V.S. Subhash, in *2022 4th International Conference on Smart Systems and Inventive Technology (ICSSIT)*. Phishing detection using machine learning algorithms (IEEE, Tirunelveli, 2022), pp. 921–924
51. T.O. Ojewumi, G.O. Ogunleye, B.O. Oguntunde, O. Folorunsho, S.G. Fashoto, N. Ogbu, Performance evaluation of machine learning tools for detection of phishing attacks on web pages. *Sci. Afr.* **16**, e01165 (2022)
52. R.E. Schapire, The strength of weak learnability. *Mach. Learn.* **5**(2), 197–227 (1990)
53. M. Alsharaiah, A. Abu-Shareha, M. Abualhaj, L. Baniata, O. Adwan, A. Al-saaidah, M. Oraiqat, A new phishing-website detection framework using ensemble classification and clustering. *Int. J. Data Netw. Sci.* **7**(2), 857–864 (2023)
54. K. Fukushima, Neocognitron: a self-organizing neural network model for a mechanism of pattern recognition unaffected by shift in position. *Biol. Cybern.* **36**(4), 193–202 (1980)

55. Y. LeCun, L. Bottou, Y. Bengio, P. Haffner, Gradient-based learning applied to document recognition. *Proc. IEEE* **86**(11), 2278–2324 (1998)
56. Y. LeCun, Y. Bengio, G. Hinton, Deep learning. *Nature* **521**(7553), 436–444 (2015)
57. S. Kiranyaz, O. Avci, O. Abdeljaber, T. Ince, M. Gabbouj, D.J. Inman, 1D convolutional neural networks and applications: a survey. *Mech. Syst. Signal Process.* **151**, 107398 (2021)
58. S. Hochreiter, J. Schmidhuber, Long short-term memory. *Neural Comput.* **9**(8), 1735–1780 (1997)
59. L.M. Rojas-Barahona, Deep learning for sentiment analysis. *Lang. Linguist. Compass* **10**(12), 701–719 (2016)
60. R.H. Alvi, M.H. Rahman, A.A.S. Khan, R.M. Rahman, Deep learning approach on tabular data to predict early-onset neonatal sepsis. *J. Inf. Telecommun.* **5**(2), 226–246 (2021)
61. D.P. Kingma, J. Ba, Adam: A method for stochastic optimization. *CoRR abs/1412.6980* (2015)
62. Z. Zhang, M.R. Sabuncu, in *32nd Conference on Neural Information Processing Systems (NeurIPS)*. Generalized cross entropy loss for training deep neural networks with noisy labels (Neural Information Processing Systems Foundation, Inc., Montreal, 2018)
63. D.M.W. Powers, Evaluation: From precision, recall and f-measure to roc, informedness, markedness & correlation. *J. Mach. Learn. Technol.* **2**(1), 37–63 (2011)
64. R. Schwartz-Ziv, A. Armon, Tabular data: deep learning is not all you need. *Inf. Fusion* **81**, 84–90 (2022)
65. B.D. Mowery, The paired t-test. *Pediatr. Nurs.* **37**(6), 320–322 (2011)

Publisher's Note

Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.